

VLASH: Real-Time VLAs via Future-State-Aware Asynchronous Inference

Jiaming Tang^{1,*} Yufei Sun^{1,3,*} Yilong Zhao⁴ Shang Yang¹ Yujun Lin²
Zhuoyang Zhang¹ James Hou^{1,6} Yao Lu² Zhijian Liu^{2,5} Song Han^{1,2}

¹MIT ²NVIDIA ³Tsinghua University ⁴UC Berkeley ⁵UCSD ⁶Caltech

<https://github.com/mit-han-lab/vlash>

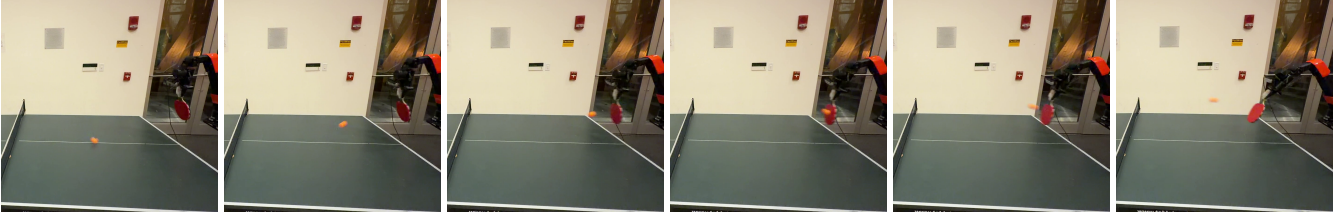


Fig. 1: VLASH enables a VLA to play ping-pong rallies with humans. Snapshots showing $\pi_{0.5}$ [1] with VLASH tracking and striking a fast-moving ping-pong ball during a rally. The robot initiates its reaction by the third frame, demonstrating low-latency response. The task requires fast reaction and smooth continuous motion, which are enabled by asynchronous inference with *future-state-awareness*. Under synchronous inference, the robot fails to achieve this dynamic interaction.

Abstract—Vision-Language-Action models (VLAs) are becoming increasingly capable across diverse robotic tasks. However, these models are typically deployed under synchronous inference, where the robot waits for model inference to complete before acting, and cannot perceive or respond to environmental changes during action execution. This not only introduces noticeable action stalls, but also significantly increases reaction latency, fundamentally limiting the applicability of VLAs to dynamic, real-time tasks. Asynchronous inference offers a promising solution to achieve continuous and low-latency control by enabling robots to execute actions and perform inference simultaneously. However, because the robot and environment continue to evolve during inference, a temporal misalignment arises between the prediction and execution intervals. This leads to significant action instability, while existing asynchronous methods either degrade accuracy or introduce runtime overhead to mitigate it. We propose VLASH, a simple yet effective method for asynchronous VLA inference that delivers smooth, accurate, and fast reaction control without architectural changes or additional runtime overhead. VLASH leverages the future execution-time state by rolling the robot state forward with the previous action chunk, thereby bridging the gap between prediction and execution. Experiments show that VLASH reduces reaction latency by up to $11.8\times$ compared to synchronous inference and consistently outperforms all asynchronous baselines in accuracy. With action quantization, it further achieves $1.5\text{--}2.0\times$ task completion speedup with minimal accuracy loss. Moreover, it empowers state-of-the-art VLAs such as $\pi_{0.5}$ to handle fast-reaction, high-precision tasks including playing ping-pong and playing whack-a-mole, where traditional synchronous inference fails.

I. INTRODUCTION

Recent advances in Vision-Language-Action models (VLAs) such as $\pi_{0.5}$ [1] and GR00T [2] have demonstrated remarkable capabilities in solving complex robotic tasks. In real-world deployment, these models are typically executed under a *synchronous inference* paradigm: the robot first

performs model inference to generate an action chunk [3], then sequentially executes the actions before initiating the next inference cycle. This sequential pipeline not only introduces action stalls, but also prevents the robot from perceiving and reacting to environmental changes during execution, significantly increasing reaction latency and limiting its ability to handle tasks requiring fast, dynamic interaction [4].

To prevent this stop-and-go behavior and reduce reaction latency, researchers have proposed *asynchronous inference* [5], [4], [6], [7]. Specifically, asynchronous inference allows the robot to execute the current action chunk while simultaneously performing inference for the next one. Because the execution duration of an action chunk is typically longer than the model inference time, the robot can immediately switch to the next chunk once the inference completes, avoiding the idle period between chunks [5], [4], [6], [7]. This design eliminates action stalls and allows the robot to perform smooth, continuous motion. Moreover, since inference is performed continuously, the robot can maintain real-time perception and thus react to environmental changes more promptly and accurately [4], [7]. In summary, asynchronous inference provides a promising way to achieve smooth, accurate, and fast reaction control for VLAs.

However, asynchronous inference faces a fundamental challenge that makes it unstable and inaccurate in practice. Since both the robot and the environment continue to evolve during inference, a *temporal misalignment* arises between the prediction interval starting when inference begins and the execution interval starting when inference finishes [4], [6]. As a result, the newly generated action misaligns with the robot’s execution-time state and environment, leading to severe instability and degraded control accuracy. For example, naive asynchronous inference reduces reaction latency but exhibits

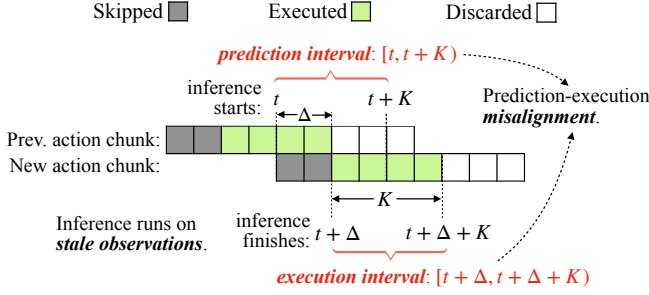


Fig. 2: **Prediction-execution misalignment in asynchronous inference.** Due to inference delay Δ , the model predicts actions for the *prediction interval* $[t, t + K)$ but they execute during the *execution interval* $[t + \Delta, t + \Delta + K)$.

unstable and laggy control performance [4]. RTC [4] mitigates this by freezing actions guaranteed to execute and inpainting the rest, but the inpainting overhead notably increases inference latency, widening the prediction-execution gap and thus degrading accuracy in latency-sensitive scenarios.

To address these challenges, we propose VLASH, a *general asynchronous inference framework for VLAs that achieves smooth, accurate, and fast reaction control without architectural changes or additional runtime overhead*. Specifically, VLASH makes the model *future-state-aware* by accurately estimating the execution-time robot state using the previously issued action chunk. Conditioned on estimated future robot states, the model is able to generate future actions in a temporally aligned way, effectively *bridging the gap between prediction and execution*. With a simple yet effective method, VLASH seamlessly integrates into existing fine-tuning pipelines and introduces no additional overhead in deployment, making asynchronous inference practical for real-time VLA systems.

We evaluate VLASH across various VLA models, including $\pi_{0.5}$ [1], GR00T N1.6 [8], and SmolVLA [5]. On simulation benchmarks [9], VLASH achieves up to 30.5% accuracy improvement compared to naive asynchronous baseline and consistently outperforms all baselines. In real-world experiments [5], VLASH consistently outperforms all asynchronous baselines in accuracy while achieving up to **11.8×** reaction speedup over synchronous inference. When combined with action quantization, it further delivers **1.5–2.0×** task completion speedup with minimal accuracy loss. Notably, VLASH enables state-of-the-art VLAs such as $\pi_{0.5}$ to perform highly dynamic tasks, including playing ping-pong rallies with a human and whack-a-mole, that are infeasible under synchronous control.

II. RELATED WORK

a) Vision-Language-Action Models (VLAs): Recent advances in Vision-Language-Action models have demonstrated remarkable capabilities in robotic manipulation by leveraging large-scale pretraining on diverse and internet-scale vision-language data. Models such as $\pi_{0.5}$ [1], GR00T [2], and others [10], [11] combine visual encoders with large language models to enable generalist robotic policies that can follow natural language instructions and generalize across tasks and

embodiments. These models are typically deployed under synchronous inference, where the robot waits for model inference to complete before executing actions, resulting in action stall and slow reaction to environmental changes [4], [6]. Our work addresses this limitation by enabling efficient asynchronous inference for VLAs.

b) Asynchronous VLA Inference: Asynchronous inference offers a promising way to eliminate action stalls and improve reaction speed of VLAs, but existing approaches still face significant barriers to adoption in VLA community. SmolVLA [5] implements naive asynchronous inference by directly switching to new action chunks, but this leads to notable prediction-execution misalignment and unstable control. Real-time Chunking (RTC) [4] mitigates this by freezing actions guaranteed to execute and inpainting the remaining actions, but the inpainting process increases the inference latency, which in turn widens the prediction-execution gap and causes the model to operate under greater state staleness, leading to accuracy degradation in latency-sensitive scenarios. Training-time RTC [12] addresses the runtime overhead of inference-time RTC by shifting the correction to training time, which supports denoising actions at two different timesteps within the same action chunk. A2C2 [6] adds additional correction heads to the model to mitigate the prediction-execution misalignment, but this also introduces runtime overhead and requires architecture changes. In contrast, our method achieves asynchronous inference through future-state-awareness by only modifying the model input at inference time, requiring no changes to the model architecture or inference procedure. We note that training-time RTC [12] and A2C2 [6] are concurrent works. RDT-2 [13] and DynamicVLA [14] reduce latency by training lightweight models that run inference at high frequency, but the smaller model size may limit the capability across diverse tasks. VLASH is model-agnostic and enables state-of-the-art VLAs to achieve low reaction latency without sacrificing capability or accuracy.

III. BACKGROUND AND MOTIVATION

a) Action chunking policy: We consider an *action chunking policy* $\pi_{\theta}(A_t | o_t, s_t)$ [3], [1], [5], where o_t is the environment observation (e.g., image, multi-view visual input), s_t is the robot state (e.g., joint positions, gripper state), and t is the controller timestep. At each timestep t , the policy generates a chunk of future actions

$$A_t = [a_t, a_{t+1}, \dots, a_{t+H-1}],$$

where H is the number of actions in the chunk. We refer to H as the *prediction horizon*.

b) Prediction and execution intervals: In practice, only the first $K \leq H$ actions from each chunk are executed before the next inference to ensure control accuracy. We denote K as the *execution horizon*. For a chunk A_t predicted at timestep t , we define the *prediction interval*

$$I_t^{\text{pred}} = [t, t + K)$$

as the time interval where the first K actions from the action chunk A_t are *planned* to be executed. During actual execution,

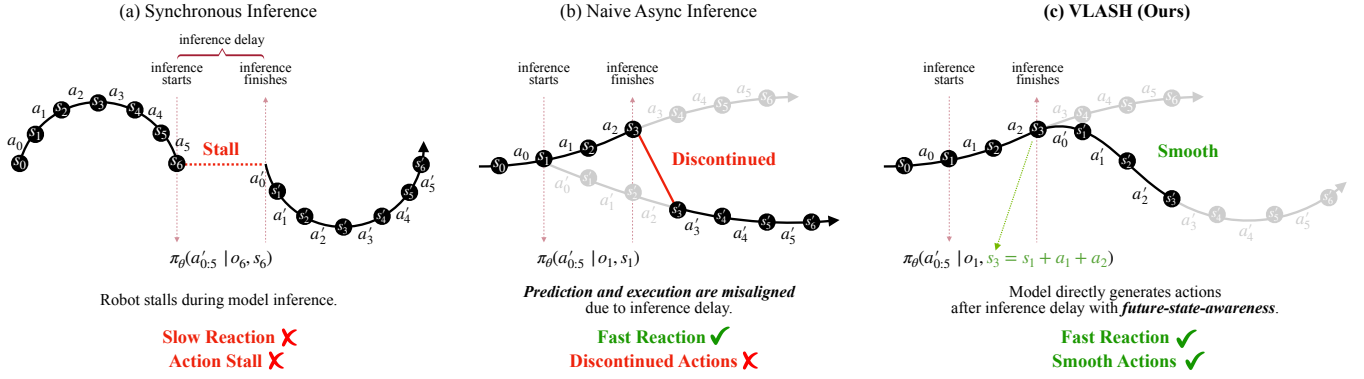


Fig. 3: **Comparison between VLASH and existing methods.** (a) Synchronous inference: the robot stalls during inference, introducing slow reactions. (b) Naive async: the model predicts based on stale state s_1 while execution begins at future state s_3 , causing misalignment and discontinuity. (c) VLASH rolls forward the robot state ($s_3 = s_1 + a_1 + a_2$) and conditions on the execution-time state, achieving fast reaction and smooth actions.

however, the K actions from A_t will start being applied later due to inference latency [5], [4].

Let $\Delta > 0$ be the inference latency measured in control steps. Then the K actions from A_t are actually executed on the robot over the *execution interval*

$$I_t^{\text{exec}} = [t + \Delta, t + \Delta + K).$$

c) Asynchronous inference and interval misalignment:

With asynchronous inference, the robot continues executing the previous action chunk while π_θ computes A_t in the background. As illustrated in Fig. 2, when $\Delta > 0$, the action chunk A_t is *planned* for the prediction interval $I_t^{\text{pred}} = [t, t + K)$ but actually *executed* over the shifted execution interval $I_t^{\text{exec}} = [t + \Delta, t + \Delta + K)$. Intuitively, the actions in A_t are not wrong for the original prediction interval $[t, t + K)$. However, under asynchronous inference, by the time they are executed, the environment and robot state have changed, so the same action sequence is applied to a different state and scene, leading to unstable and discontinuous behavior [4], [6].

IV. VLASH

A. Future State Awareness

In asynchronous inference, the robot keeps moving while the VLA performs a forward pass, so the state at inference start generally differs from the state at which the new actions actually begin execution. Our key idea is to make the policy *future-state-aware*: instead of conditioning on the current robot state s_t , we condition on the robot state at the beginning of the next execution interval $s_{t+\Delta}$.

Although the future environment observation is unknown, the robot state at the beginning of the execution interval $s_{t+\Delta}$ is determined by the current robot state s_t and the actions executed during the inference delay $a_{t:t+\Delta-1}$. As shown in Fig. 3(c), when inference for the new chunk starts at state s_1 , the robot will still execute the remaining actions a_1, a_2 from the previous chunk before the new chunk is ready to take over. Since the actions a_1, a_2 are already known, we can *roll the state forward* under them to estimate the execution-time state.

In Fig. 3(c), this corresponds to computing $s_3 = s_1 + a_1 + a_2$, which gives the robot state at the start of the execution interval. For *delta* actions, the future state is estimated by accumulating the action deltas onto the current state; for *absolute* actions, the last action in the executed sequence directly serves as the estimated future state. During the forward pass, VLASH feeds both the current environment observation o_1 and this rolled-forward future state s_3 into the VLA. In this way, the model generates actions for the state at the execution-time rather than for the stale state at inference start, *bridging the gap between prediction and execution in terms of robot state*. While the future environment is still unknown, this mechanism mirrors how humans act under reaction delays: we react to the world with slightly outdated visual input, but use our internal body state to anticipate what we will do when the action actually takes effect. Thus, humans naturally compensate for such reaction delay, and we expect VLAs to possess the same capability.

B. Fine-tuning with Offsets to States and Actions

State-of-the-art VLAs universally take robot state as input alongside visual observations [1], [5], [2], since identical images (e.g., a wrist camera facing a blank wall) can correspond to vastly different robot states or positions, making visual input alone insufficient for accurate action prediction. The future-state-awareness also assumes that the VLA is able to leverage the rolled-forward robot state properly. However, we find that current VLAs tend to *under-utilize the robot state* in practice, and therefore fail to leverage the rolled-forward future state effectively. Therefore, simply feeding a future robot state at test time is insufficient to achieve accurate and stable asynchronous control. Since large VLAs are almost always fine-tuned on downstream data before deployment, we design a training augmentation that can be seamlessly integrated into the standard fine-tuning stage with *no additional overhead*. We keep the architecture and fine-tuning pipeline unchanged, and only modify how training samples are constructed.

Concretely, given a trajectory $\{(o_t, s_t, a_t)\}$, standard fine-tuning trains the model to predict the action chunk $a_{t:t+H-1}$

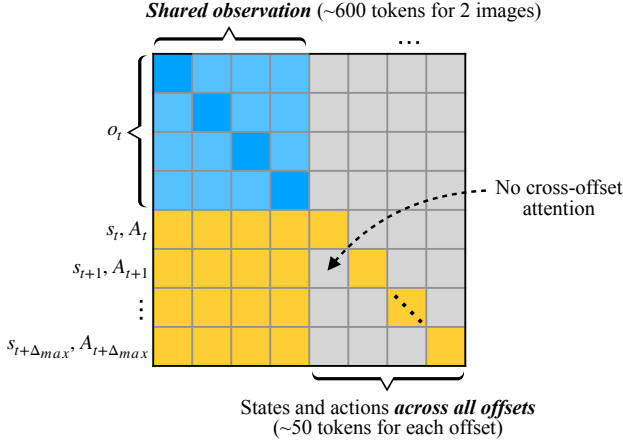


Fig. 4: **Attention pattern for efficient fine-tuning with shared observation.** We pack one shared observation o_t and multiple offset branches ($s_{t+\delta}, A_{t+\delta}$) into a single sequence. Blue cells indicate allowed attention, while gray cells indicate masked attention.

from (o_t, s_t) . We instead apply a simple temporal-offset augmentation with two key steps:

- (i) **Offset state and action together.** We sample a random offset δ from a predefined range (e.g., $\delta \in \{0, \dots, \Delta_{\max}\}$) and construct training targets from the future state $s_{t+\delta}$ and future action chunk $a_{(t+\delta):(t+\delta+H-1)}$ on the same trajectory.
- (ii) **Fix the environment observation.** For each timestep t , we always use the same visual input o_t when varying δ . Therefore, the model is trained to predict $a_{(t+\delta):(t+\delta+H-1)}$ from the pair $(o_t, s_{t+\delta})$.

Under this scheme, the same image o_t can correspond to different ground-truth actions depending on the offset robot state $s_{t+\delta}$. To fit the data, the VLA is forced to attend to the state input rather than overfitting purely to visual features. In particular, it learns to interpret $s_{t+\delta}$ as a meaningful future state for action selection. We train over a range of offsets δ rather than a single fixed offset because, in practice, the same VLA may be deployed on hardware with different compute budgets, leading to different inference delays Δ . This makes the model *compatible with different inference delays* while preserving performance in the synchronous case. With asynchronous inference, we can then feed the rolled-forward execution-time state together with the current observation, and the fine-tuned VLA naturally leverages this future state to produce actions that are aligned and stable over the execution interval. At deployment, we simply select the smallest offset δ that is greater than the measured inference latency Δ .

C. Efficient Fine-tuning with Shared Observation

The temporal-offset augmentation creates multiple state-action pairs for the same observation o_t , and it admits two implementations. The *basic implementation* directly follows the sampling procedure described above: each training example draws a single random offset δ , and the VLA runs independently on $(o_t, s_{t+\delta}, A_{t+\delta})$, treating different offsets

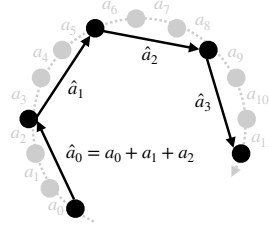


Fig. 5: **Action quantization for efficient execution.** Adjacent fine-grained actions (gray) are grouped into coarser macro-actions (black) to accelerate robot motion. Each macro-action summarizes q consecutive actions, e.g., $\hat{a}_0 = a_0 + a_1 + a_2$ for $q=3$. For non-integer q , we re-sample the trajectory via linear interpolation.

of the same observation as separate training examples. This implementation is completely plug-and-play and can be seamlessly integrated into existing VLA fine-tuning pipeline. However, it repeatedly encodes the same observation o_t for every sampled offset, leaving substantial room for further efficiency gains.

Instead, our *efficient implementation* exploits the fact that all offsets share the same observation o_t and designs an efficient attention pattern that reuses the observation tokens across offsets in a single pass (Fig. 4). Concretely, rather than sampling one offset per example, we pack one observation and all offset branches $\delta \in \{0, \dots, \Delta_{\max}\}$ into a single sequence:

$$[o_t, (s_t, A_t), (s_{t+1}, A_{t+1}), \dots, (s_{t+\Delta_{\max}}, A_{t+\Delta_{\max}})],$$

where each $(s_{t+\delta}, A_{t+\delta})$ corresponds to one temporal offset. We then apply a block-sparse self-attention mask with the following structure:

- All **observation tokens** (e.g., image tokens from two views and language tokens, about ~ 600 tokens in total for $\pi_{0.5}$) can attend to each other, as in standard VLA fine-tuning.
- For each **offset branch**, the state-action tokens $(s_{t+\delta}, A_{t+\delta})$ can attend to all observation tokens and to tokens within the same offset, but *cannot* attend to tokens from other offsets.

This attention map, illustrated in Fig. 4, makes different offsets condition on a shared observation while remaining independent of each other. For each offset branch, the **positional encodings** of $(s_{t+\delta}, A_{t+\delta})$ are assigned to start at the same index, equal to the length of observation tokens. From the model’s perspective, this is equivalent to training on multiple $(o_t, s_{t+\delta}, A_{t+\delta})$ examples that share the same o_t , but we only encode o_t *once*.

For $\pi_{0.5}$, an observation with two images and language tokens corresponds to ~ 600 tokens in total, while one state and action chunk are about ~ 50 tokens [1]. Packing $N_\delta = 5$ offsets into a single sequence therefore increases the token length by only $\sim 31\%$ (from 650 to 850 tokens) relative to a single-offset sequence, while the number of effective training trajectories becomes $5\times$ larger. In practice, under the same *effective batch size* as standard fine-tuning, this method can significantly

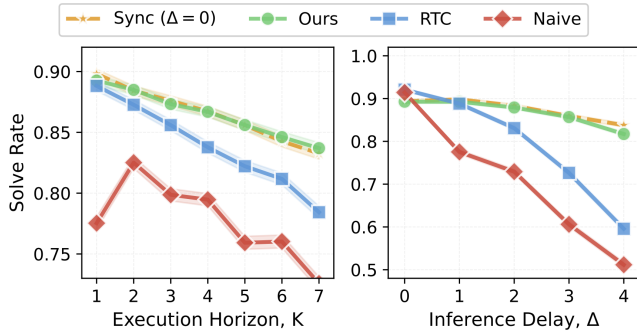


Fig. 6: **Performance on Kinetix benchmark.** We evaluate the success rate under different execution horizons K and inference delays Δ . **Left:** Fixed inference delay $\Delta = 1$ with varying execution horizon K . **Right:** Execution horizon adapts to inference delay, i.e., $K = \max(\Delta, 1)$, with varying Δ . For the **Sync** baseline, inference delay is always $\Delta = 0$, but the execution horizon K follows the same settings as other baselines for fair comparison.

improve training efficiency by reusing each observation across multiple offset targets in a single pass.

D. Action Quantization

Highly dynamic tasks such as playing ping-pong or whack-a-mole require the robot to not only react fast but also move fast enough to complete the task. With asynchronous inference, model inference is effectively hidden by action execution, so the system speed is limited by how fast the robot executes the action sequence. To accelerate the physical motion, we adopt *action quantization* [15], [16], which groups adjacent fine-grained actions into coarser macro-actions. For a quantization factor q , each macro-action summarizes q consecutive actions; for delta actions, this is simply $\hat{a}_i = a_{iq} + a_{iq+1} + \dots + a_{(i+1)q-1}$, moving the robot from the start state of a_{iq} to the end state of $a_{(i+1)q-1}$ in a single step (Fig. 5). When q is non-integer, we use linear interpolation to re-sample the original trajectory into a shorter sequence. This offers a tunable speed-accuracy trade-off: larger q yields faster but coarser motion. In practice, we select task-dependent quantization factors that substantially reduce execution time while maintaining accuracy close to the unquantized policy. Since VLASH achieves smooth control without action stalls or additional runtime overhead, it consistently outperforms synchronous and asynchronous baselines in accuracy when combined with action quantization under the same speedup (Sec. V-B).

V. EXPERIMENTS

We evaluate VLASH across both simulated and real-world settings, each covering: (1) *general manipulation* (LIBERO in simulation; pick&place, stacking, sorting in real-world), which validates that VLASH **maintains or improves accuracy** on general VLA tasks; and (2) *dynamic interaction* (Kinetix in simulation; ping-pong, whack-a-mole in real-world), which demonstrates that VLASH **unlocks fast-reaction tasks** previously infeasible with synchronous inference.

TABLE I: **Performance on LIBERO benchmarks with different inference delays.** We evaluate $\pi_{0.5}$ [1] and SmolVLA [5] across four LIBERO sub-benchmarks (Spatial, Object, Goal, Long) under various inference delays (0 to 3 steps). **SR:** average success rate (%).

Method	Delay	Success Rate (%)				Avg.
		Spatial	Object	Goal	Long	SR
$\pi_{0.5}$ [1]						
Sync	0	97.3	99.6	96.7	93.5	96.8
VLASH (Async)	1	98.8	99.2	96.7	94.4	97.2
	2	97.5	99.2	97.3	94.6	97.1
	3	94.4	98.8	93.3	91.9	94.6
$SmolVLA$ [5]						
Sync	0	81.3	92.9	85.8	55.8	79.0
VLASH (Async)	1	80.0	92.9	82.3	53.1	77.1
	2	78.5	92.1	86.9	55.0	78.1
	3	79.8	94.2	87.5	54.8	79.1

A. Simulated Evaluation

We evaluate VLASH on simulated robotic manipulation benchmarks, including Kinetix [9] and LIBERO [17].

1) Kinetix:

a) *Experimental Setup:* Kinetix [9] is a highly dynamic simulated robotic manipulation benchmark that demands asynchronous execution to handle rapidly changing environments. The tasks are designed to test dynamic reaction capabilities, including throwing, catching, and balancing.

Following the setup in RTC [4], we train action chunking flow policies with a prediction horizon of $H = 8$ and a 4-layer MLP-Mixer [18] architecture for 32 epochs. We report average success rates across 12 tasks, each evaluated with 1,024 rollouts per data point, under simulated delays ranging from 0 to 4 steps. To estimate the future robot state, we roll the environment forward in the Kinetix simulator using the generated actions. The simulation noise used for this rollforward is sampled independently from the noise used during actual execution, so the estimated state is approximate. We then extract the robot-state components from the resulting state vector. We compare against three baselines: **Sync** (serves as an optimal baseline with inference delay set to 0), Naive Async, and RTC [4].

b) *Results:* As shown in Fig. 6, VLASH tracks the synchronous upper bound closely across execution horizons, while other baselines drop more noticeably as the execution horizon increases. When the inference delay increases, VLASH remains robust and consistently achieves high success rates, while RTC degrades rapidly and the Naive Async baseline collapses under larger delays. Notably, at inference delay of 4 steps, VLASH achieves 81.7% success rate compared to only 51.2% for Naive Async, which is a substantial 30.5% accuracy improvement. Overall, VLASH effectively mitigates prediction-execution misalignment, delivering high success rates under asynchronous operation.

2) LIBERO:

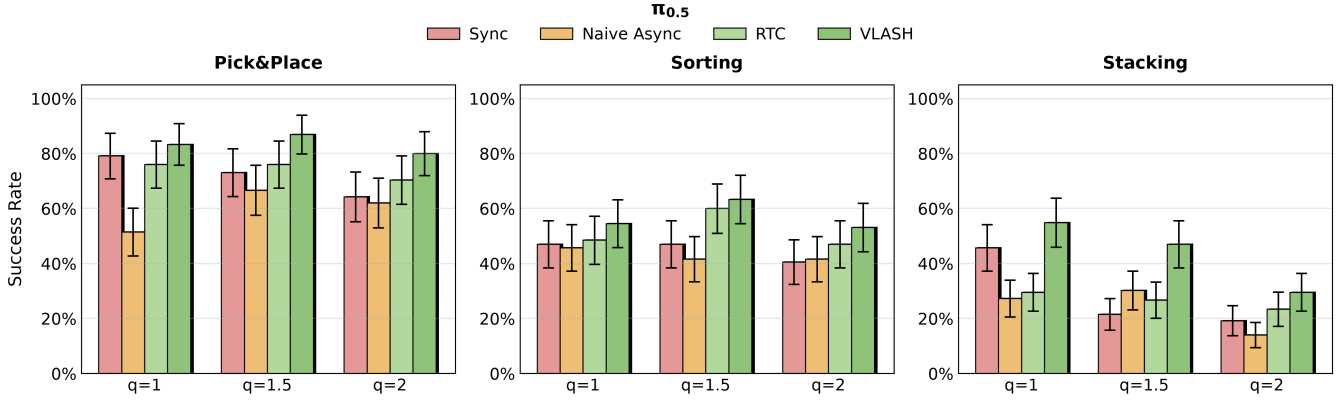


Fig. 7: **Real-world evaluation with $\pi_{0.5}$.** Success rate across three tasks with different inference methods and action quantization factors q . Each task is evaluated over 20 rollouts (up to 3 attempts per rollout). Success rate is computed as the number of successes over total attempts, with error bars showing binomial standard error.

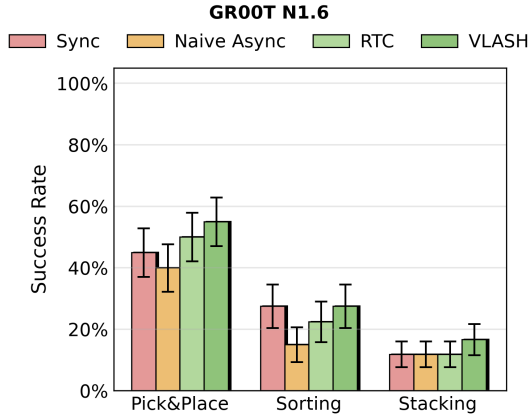


Fig. 8: **Real-world evaluation with GR00T N1.6.** Success rate across three tasks at $q=1$. Same evaluation protocol as $\pi_{0.5}$. We only report $q=1$ as higher quantization factors yield limited success rates across all methods, making it difficult to draw meaningful comparisons.

a) Experimental Setup: We evaluate on the LIBERO benchmark [17], one of the popular benchmarks for evaluating VLA, which includes 4 different sub-benchmarks (Spatial, Object, Goal, and Long) that contain 10 tasks each. We evaluate on 2 state-of-the-art VLAs: $\pi_{0.5}$ [1] and SmolVLA [5]. We report the performance by fine-tuning all models on the training dataset for 30K iterations with a batch size of 32. Following the setup in $\pi_{0.5}$ [1], we set the execution horizon to $K = 5$ [19]. Since LIBERO tasks involve slowly changing environments with mild state transitions, different asynchronous methods behave similarly. Therefore, we focus our comparisons with synchronous inference. In simulation benchmarks, synchronous inference does not suffer from the motion jitter and discontinuities caused by action stalls, making it a stronger baseline than in real-world settings. Asynchronous inference thus operates at an inherent disadvantage here due to state staleness. This benchmark thus serves to

validate that VLASH can maintain strong accuracy across different VLA architectures under large inference delays.

b) Results: As shown in Table 1, VLASH demonstrates strong performance across both $\pi_{0.5}$ and SmolVLA under various inference delays. With small delays, VLASH maintains accuracy comparable or even better than synchronous inference on $\pi_{0.5}$, achieving 97.2% and 97.1% average SR compared to 96.8% for Sync. At larger delays, accuracy decreases slightly but remains competitive, demonstrating that the future-state-awareness effectively preserves accuracy under asynchronous execution. On SmolVLA, VLASH at delay 3 achieves 79.1% SR, slightly outperforming the synchronous baseline’s 79.0% SR, further validating its generalization across different VLA architectures.

B. Real-World Evaluation

To evaluate VLASH in real-world settings, we deploy $\pi_{0.5}$ [1] and GR00T N1.6 [8] on two robotic platforms: the Galaxea R1 Lite [20] and the LeRobot SO-101 [21]. The R1 Lite is a dual-arm robot equipped with two 7-DOF arms from Galaxea [22]. The SO-101 is a 6-DOF collaborative robotic arm from LeRobot [23]. For $\pi_{0.5}$, we apply a projection layer to map the robot state into an embedding, bypassing the tokenizer instead of incorporating it into the language prompt in the original implementation. We design our real-world experiments to evaluate three key aspects: (1) Success rate: the task success rate across manipulation tasks under different inference methods; (2) Speed-accuracy trade-off and robustness: how well accuracy is preserved when applying action quantization for faster execution; and (3) Reaction speed: the latency to react to dynamic changes in the environment.

1) Success Rate and Robustness:

a) Experimental Setup: Following the setup in SmolVLA [5], we evaluate $\pi_{0.5}$ [1] and GR00T N1.6 [8] on three manipulation tasks that test different aspects of robotic control. We set the execution horizon to $K = 24$ steps at 30 Hz. All experiments are conducted on a *laptop* with NVIDIA RTX 5090 GPU, with an inference delay of 2 steps.

TABLE II: **Reaction speed comparison across GPUs.** We report the maximum reaction latency of $\pi_{0.5}$ [1] with 1 image input, $K = 25$ at 50 Hz; the execution duration D is 500 ms. With inference latency L , the maximum reaction latency is $D + 2L$ for Sync and $2L$ for Async with back-to-back inference. Both VLASH and RTC enable `torch.compile` for optimized inference speed.

	H100	RTX 5090	RTX 4090
Inference Latency L (ms)			
Sync, VLASH	23.2	29.4	32.3
RTC	48.4	49.6	54.2
Max Reaction Latency (ms)			
Sync	546.4	558.8	564.6
RTC	96.8	99.2	108.4
VLASH	46.4	58.8	64.6
Speedup			
vs. Sync	11.8×	9.5×	8.7×
vs. RTC	2.1×	1.7×	1.7×

These tasks serve to verify that our method does not hurt general manipulation capability, and to demonstrate that asynchronous inference can *improve not only reaction speed but also accuracy* by enabling smoother motion and eliminating the jitter caused by action stalls [4]. On our robotic platforms, we evaluate three tasks:

- **Pick and Place:** pick up a cube from varying starting positions and place it into a fixed box;
- **Stacking:** pick up an orange cube and stack it on top of a blue cube, where both cubes’ initial positions vary across episodes;
- **Sorting:** sort cubes by color, placing the orange cube in the right box and the blue cube in the left box, with cube positions varying across episodes.

For each task, we conduct 20 rollouts per method. Each rollout allows up to 3 attempts, and we report the success rate computed as the number of successes divided by the total number of attempts across all rollouts, with error bars computed using binomial standard error. We compare synchronous inference, naive asynchronous inference, RTC [4] and VLASH across these tasks.

b) Results: As shown in Fig. 7 and Fig. 8, VLASH consistently achieves the highest success rate across all tasks and action quantization factors, outperforming synchronous, naive asynchronous, and RTC [4] baselines. Moreover, VLASH is more robust to action quantization than all baselines. On pick&place and sorting tasks, VLASH at $q=2$ matches or exceeds the success rate of synchronous inference at $q=1$, enabling $2\times$ speedup with no accuracy loss. On the stacking task, which requires more precise manipulation than the other two tasks, VLASH at $q=1.5$ still maintains comparable accuracy to Sync at $q=1$. VLASH also outperforms RTC [4] across all settings, as RTC’s inference-time overhead increases the inference latency and widens the prediction-execution gap, causing it to operate under greater state staleness than ours. In practice, a quantization factor of q yields roughly $q\times$ task

TABLE III: **Real-world evaluation on dynamic interactive tasks.** We evaluate $\pi_{0.5}$ on ping-pong (single-return hit rate over 20 trials) and whack-a-mole (average score over 5 rounds) with $q=2$.

Method	Ping-Pong Hit Rate $\uparrow$	Whack-a-Mole Avg. Score $\uparrow$
Sync	0/20 (0%)	3.2
Naive Async	0/20 (0%)	8.6
RTC	0/20 (0%)	11.0
VLASH (ours)	11/20 (55%)	28.8

speedup when accuracy is preserved, so we can select the largest q that does not degrade accuracy compared to Sync. In our experiments, $q \in [1.5, 2]$ achieves **1.5–2.0×** speedup without degrading accuracy, adjustable based on the precision requirements of each task.

2) Reaction Speed:

a) Experimental Setup: To evaluate the reaction speed improvement of asynchronous inference, we compare the *maximum reaction latency*, i.e., the worst-case delay from an environmental change to the first executed action informed by an observation of that change, between synchronous and asynchronous inference across different hardware configurations. In the worst case, the change occurs right after an observation is captured: synchronous inference must complete the ongoing inference (latency L), execute the entire action chunk (duration D), and run one more inference before reacting, giving $D + 2L$; asynchronous inference with back-to-back inference captures the change at the next inference start and reacts when that inference completes, giving $2L$. Following the setup in $\pi_{0.5}$ [1], we set the execution horizon to $K = 25$ for synchronous inference and a control frequency of 50 Hz [1], [4], resulting in an execution duration of approximately 0.5 seconds per action chunk. We measure the model inference latency of $\pi_{0.5}$ on three different GPUs: H100, RTX 5090, and RTX 4090, enabling CUDA Graphs and kernel fusion optimization for minimal latency. We further evaluate VLASH on two highly dynamic interactive tasks using a *laptop* with NVIDIA RTX 5090 GPU:

- **Ping-pong:** The robot plays rallies with a human player. The human player serves the ball; the robot must track the incoming ball, react in time, and strike it back. We report the single-return hit rate over 20 trials, where each trial consists of one serve from the human.
- **Whack-a-mole:** The robot must quickly detect and strike randomly activated targets on a whack-a-mole board. Each round has a 30-second time limit and terminates early after 3 missed strikes. We report the average score (number of successful hits) per round over 5 rounds.

Due to the highly dynamic nature of both tasks, we set the action quantization factor to $q=2$ for all methods, as this yields the best performance across all baselines by enabling faster robot motion to match the task dynamics.

b) Results: As shown in Table II, asynchronous inference significantly reduces the maximum reaction latency

TABLE IV: **Fine-tuning efficiency.** Original (without offset augmentation) vs VLASH (with offset augmentation and shared observation) on LIBERO with $\pi_{0.5}$ [1]. Training on 4×H100 GPUs using DDP, with effective batch size 16 per GPU (total 64). We report average LIBERO scores at different training steps. Both evaluated under synchronous inference.

Method	Time/Step (ms)	Fine-tuning Steps		
		10K	20K	30K
Original	420.99	94.1	97.1	96.8
VLASH	129.29	87.1	94.4	96.6
Speedup	3.26×	-	-	-

compared to synchronous inference, achieving up to **11.8×** speedup. On the dynamic interactive tasks (Table III), VLASH enables the robot to achieve **55%** single-return hit rate in ping-pong and an average score of **28.8** in whack-a-mole, whereas synchronous inference largely fails on both tasks due to excessive reaction latency. To the best of our knowledge, we are the first to *demonstrate a VLA successfully playing ping-pong rallies with a human*. A video demonstration is available in the supplementary material.

C. Fine-tuning Efficiency

a) Experimental Setup: We evaluate the training efficiency gains from our efficient fine-tuning with shared observation approach. A key consideration is that training with multiple temporal offsets using shared observation effectively increases the *effective batch size* by a factor equal to the number of offsets. Therefore, we compare our method against standard fine-tuning under the same *effective batch size* to ensure a fair comparison. Specifically, we conduct experiments on the LIBERO benchmark using $\pi_{0.5}$ [1] trained on 4×H100 GPUs with DDP. For our method, we use $\Delta_{\max} = 3$ with a physical batch size of 4 per GPU, resulting in an effective batch size of 16 per GPU and 64 in global. The standard baseline uses a physical batch size of 16 per GPU to match this effective batch size. Both methods are trained for 10K, 20K, and 30K iterations, and we report the average success rate across all LIBERO tasks. We also measure the training time per forward-backward pass to quantify the speedup.

b) Results: As shown in Table IV, VLASH converges more slowly in the early stages but ultimately achieves comparable accuracy to standard fine-tuning. Although more training steps are needed for convergence, each step is significantly faster, achieving a 3.26× speedup per step. This efficiency gain comes from encoding the shared observation only once and reusing it across all temporal offsets. Furthermore, since both methods are evaluated under synchronous inference, these results also demonstrate that VLASH does not hurt the original synchronous performance of the model.

VI. CONCLUSION

We present VLASH, a general and efficient framework for enabling asynchronous inference in Vision-Language-Action models. By making the policy future-state-aware

through simple state rollforward, VLASH effectively bridges the prediction-execution gap that has hindered asynchronous control. Experiments on both simulated and real-world benchmarks demonstrate that VLASH achieves smooth, accurate, and fast-reaction control, **reducing reaction latency by up to 11.8×** while **consistently matching or surpassing the accuracy of synchronous inference**. Moreover, we demonstrate that VLAs can perform highly dynamic tasks such as playing ping-pong rallies with humans. We hope these results will inspire future research toward extending VLAs to more dynamic and physically interactive domains.

REFERENCES

- [1] P. Intelligence, K. Black, N. Brown, J. Darphinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai *et al.*, “ $\pi_{0.5}$: a vision-language-action model with open-world generalization,” *arXiv preprint arXiv:2504.16054*, 2025.
- [2] J. Bjorck, F. Castañeda, N. Cherniadev, X. Da, R. Ding, L. Fan, Y. Fang, D. Fox, F. Hu, S. Huang *et al.*, “GR00T N1: An open foundation model for generalist humanoid robots,” *arXiv preprint arXiv:2503.14734*, 2025.
- [3] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, “Learning fine-grained bimanual manipulation with low-cost hardware,” *arXiv preprint arXiv:2304.13705*, 2023.
- [4] K. Black, M. Y. Galliker, and S. Levine, “Real-time execution of action chunking flow policies,” *arXiv preprint arXiv:2506.07339*, 2025.
- [5] M. Shukor, D. Aubakirova, F. Capuano, P. Kooijmans, S. Palma, A. Zoutine, M. Aractingi, C. Pascal, M. Russi, A. Marafioti *et al.*, “Smolvla: A vision-language-action model for affordable and efficient robotics,” *arXiv preprint arXiv:2506.01844*, 2025.
- [6] K. Sendai, M. Alvarez, T. Matsushima, Y. Matsuo, and Y. Iwasawa, “Leave no observation behind: Real-time correction for vla action chunks,” *arXiv preprint arXiv:2509.23224*, 2025.
- [7] Y. Ma, Y. Zhou, Y. Yang, T. Wang, and H. Fan, “Running vlars at real-time speed,” *arXiv preprint arXiv:2510.26742*, 2025.
- [8] NVIDIA, “NVIDIA Isaac GR00T N1.6,” <https://huggingface.co/nvidia/GR00T-N1.6-3B>, 2025.
- [9] M. Matthews, M. Beukman, C. Lu, and J. Foerster, “Kinetic: Investigating the training of general agents through open-ended physics-based control tasks,” in *International Conference on Learning Representations*, 2025. [Online]. Available: <https://openreview.net/forum?id=zCxGCdzreM>
- [10] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter *et al.*, “ π_0 : A vision-language-action flow model for general robot control,” *arXiv preprint arXiv:2410.24164*, 2024.
- [11] M. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn, “Openvla: An open-source vision-language-action model,” *arXiv preprint arXiv:2406.09246*, 2024.
- [12] K. Black, A. Z. Ren, M. Equi, and S. Levine, “Training-time action conditioning for efficient real-time chunking,” *arXiv preprint arXiv:2512.05964*, 2025.
- [13] S. Liu, B. Li, K. Ma, L. Wu, H. Tan, X. Ouyang, H. Su, and J. Zhu, “Rdt2: Exploring the scaling limit of uni data towards zero-shot cross-embodiment generalization,” *arXiv preprint arXiv:2602.03310*, 2026.
- [14] H. Xie, B. Wen, J. Zheng, Z. Chen, F. Hong, H. Diao, and Z. Liu, “Dynamicvla: A vision-language-action model for dynamic object manipulation,” *arXiv preprint arXiv:2601.22153*, 2026.
- [15] Figure AI, “Helix accelerating real-world logistics,” <https://www.figure.ai/news/helix-logistics>, 2025.
- [16] L. Guo, Z. Xue, Z. Xu, and H. Xu, “Demospeedup: Accelerating visuomotor policies via entropy-guided demonstration acceleration,” *arXiv preprint arXiv:2506.05064*, 2025.
- [17] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, “Libero: Benchmarking knowledge transfer for lifelong robot learning,” *arXiv preprint arXiv:2306.03310*, 2023.
- [18] I. O. Tolstikhin, N. Houlsby, A. Kolesnikov, L. Beyer, X. Zhai, T. Unterthiner, J. Yung, A. Steiner, D. Keysers, J. Uszkoreit *et al.*, “Mlp-mixer: An all-mlp architecture for vision,” *Advances in neural information processing systems*, vol. 34, pp. 24 261–24 272, 2021.

- [19] D. Driess, J. T. Springenberg, B. Ichter, L. Yu, A. Li-Bell, K. Pertsch, A. Z. Ren, H. Walke, Q. Vuong, L. X. Shi *et al.*, “Knowledge insulating vision-language-action models: Train fast, run fast, generalize better,” *arXiv preprint arXiv:2505.23705*, 2025.
- [20] Galaxea AI Technology Co., Ltd., “R1 lite,” <https://galaxea-ai.com/cn/products/R1-Lite>, 2025.
- [21] Hugging Face, Inc., “So-101,” <https://huggingface.co/docs/lerobot/en/so101>, 2025.
- [22] Galaxea AI Technology Co., Ltd., “Galaxea ai,” <https://galaxea-ai.com/cn>, 2025.
- [23] R. Cadene, S. Alibert, A. Soare, Q. Gallouedec, A. Zouitine, S. Palma, P. Kooijmans, M. Aractingi, M. Shukor, D. Aubakirova, M. Russi, F. Capuano, C. Pascal, J. Choghari, J. Moss, and T. Wolf, “Lerobot: State-of-the-art machine learning for real-world robotics in pytorch,” <https://github.com/huggingface/lerobot>, 2024.

APPENDIX I EXPERIMENTAL DETAILS

We present the detailed training hyperparameters used for fine-tuning VLAs in our experiments in Table V. For all experiments on LIBERO benchmarks and real-world tasks, we use same hyperparameters to ensure fair comparison across different methods and models. These hyperparameters are carefully tuned to balance training stability and convergence speed while preventing overfitting on the downstream tasks.

TABLE V: **Training hyperparameters for fine-tuning VLAs.** We use these hyperparameters for fine-tuning $\pi_{0.5}$ and SmolVLA on LIBERO and real-world tasks.

Hyperparameter	Value
<i>Training Configuration</i>	
Batch Size	32
Training Steps	30,000
<i>Optimizer (AdamW)</i>	
Learning Rate	5e-5
Betas	[0.9, 0.95]
Weight Decay	1e-10
<i>Learning Rate Scheduler</i>	
Type	Cosine Decay with Warmup
Warmup Steps	1,000
Peak Learning Rate	5e-5
Decay Learning Rate	2.5e-6
Decay Steps	30,000

APPENDIX II SUPPLEMENTARY DEMO VIDEO

We provide comprehensive video demonstrations comparing our method against synchronous and naive asynchronous baselines across various real-world manipulation tasks. All demonstrations are conducted using $\pi_{0.5}$ [1] deployed on a laptop with NVIDIA RTX 5090 GPU, achieving an inference frequency of 15 Hz.

We showcase the following tasks in the supplementary materials:

- **Ping-pong:** Interactive rallies with a human player, demonstrating rapid reaction capabilities.
- **Whack-a-mole:** Fast-response game requiring quick detection and precise striking motions.
- **Pick and place:** Standard manipulation task showing smooth motion control.
- **Folding clothes:** Complex manipulation requiring coordinated movements.

We compare three inference modes: synchronous inference, naive asynchronous inference, and VLASH. Additionally, we demonstrate the effects of action quantization, showing how our method can achieve further speedups while maintaining task performance.

The video demonstrations clearly show that VLASH produces noticeably smoother motions and faster task completion compared to both synchronous and naive asynchronous baselines. The synchronous baseline often exhibits stuttering

behavior due to action stalls, while naive asynchronous inference suffers from prediction-execution misalignment that leads to erratic movements. In contrast, VLASH maintains fluid motion throughout task execution while achieving significant speedup. We encourage readers to view the video to appreciate the dynamic performance improvements of our approach.

APPENDIX III ARCHITECTURAL MODIFICATIONS

A key advantage of VLASH is that it requires *no architectural modifications* to achieve effective performance across diverse VLA models. Since all current VLA models accept robot state inputs, VLASH can be applied directly by simply offsetting the state information during fine-tuning to account for inference delay. This straightforward approach enables the model to learn the temporal alignment between delayed observations and corresponding actions without any changes to the model architecture.

For standard VLA architectures like π_0 [10] and SmolVLA [5], which incorporate a state projection layer to embed proprioceptive state vectors into continuous representations before feeding them into the transformer backbone, VLASH integrates seamlessly and achieves excellent results out of the box.

We further note that VLASH also works directly with $\pi_{0.5}$ [1] without modifications, as demonstrated in our experiments in Table I. However, $\pi_{0.5}$ employs a unique design that converts numerical state values into text tokens and appends them to the language prompt. This text-based encoding forces numerical state values through tokenization and one-hot encoding, disrupting their inherent numerical structure and making it more challenging for the model to learn from state information. For such architectures, we find that adding a lightweight state projection like the design of π_0 and injecting the resulting embeddings back into their original positions can further enhance smoothness and stability. A simpler alternative is to incorporate the projected state embeddings into the AdaRMSNorm layers as conditioning signals alongside timestep embeddings. While entirely optional (and VLASH already performs well without it), this small architectural enhancement consistently improves control smoothness for $\pi_{0.5}$. Importantly, the additional parameters introduced by this state projection layer are negligible: it consists only of a linear mapping from the state dimension to the hidden dimension. Moreover, because it is zero-initialized, it completely preserves the pretrained model’s performance during the initial stages of fine-tuning.